M782 Project: Proof for System F+ in Agda

Xiao Chen, Joshua Crotts, Lillie Donato

April 27, 2026

1 Introduction

System F, which combines the *simply-typed lambda calculus* (STLC) with *parametric polymorphism*, was first defined by Jean-Yves Girard in his PhD thesis to formalize functions that can be proven to be total in second-order logic [4, 5].

Specifically, System F allows us to quantify over types using type variables and instantiate these type variables with *any* concrete type. For example, we can define the identity function $\Lambda \,.\, \lambda \,.\, (\texttt{var}\ 0)$ of polymorphic type $\forall.\, (\texttt{TVar}\ 0) \to (\texttt{TVar}\ 0)$, and instantiate $(\texttt{TVar}\ 0)$ to `Nat`. To formalize the semantics, the De Bruijn indices [3] are applied to both term variables and type variables.

This project revolves around a variation of System F, which we call System F+, supporting operations from STLC (with De Bruijn indices), parametric polymorphism, lists, and fixed-point operators. We formalize System F+ in Agda and prove type safety using its progress and preservation lemmas. The remainder of this document is organized as follows.

- In section 2 we describe System F+, including its syntax, type system, and operational semantics.
- In section 3 we discuss the type safety of System F+. To complete the proof, we focus on its progress and preservation theorems (as well as their lemmas) to showcase that type safety is an invariant preserved by the operational semantics.
- In section 4 we propose an interpreter in Racket that automatically add type annotations for expressions in s-expression forms.

2 A Formalism for System F+

A formalism of System F+ consists of its syntax, typing judgments, and reduction rules.

2.1 Syntax for System F+

Types	τ	$::=$	`Unit`	Unit Type
			`Nat`	Natural Numbers
			`Bool`	Boolean
			`TVar` $\mathbb{N}$	Type Variables with De Bruijn Indices
			$\tau_1 \times \tau_2$	Pair Type
			$\tau_1 \oplus \tau_2$	Sum Type
			$\tau_1 \to \tau_2$	Function Type
			$\forall.\, \tau_0$	Polymorphic Type
			`List` τ_0	List Type

Terms	e ::=	$\texttt{unit}$	Unit
		$\texttt{var}\ \mathbb{N}$	Variables
		$\texttt{true}$	Bollean: True
		$\texttt{false}$	Boolean: False
		$\texttt{zero}$	Natural Number: Zero
		$\texttt{suc}\ e_0$	Natural Number: Successor
		$\texttt{suc}\ e_0$	Natural Number: Predecessor
		$\texttt{zero?}\ e_0$	Zero Predicate
		$\lambda\,.\,e_0$	λ-Abstraction
		$\texttt{app}\ e_1\ e_2$	Application
		$\texttt{if}\ e_0\ e_1\ e_2$	Conditional Branching
		$\langle e_1, e_2 \rangle$	Pair
		$\texttt{fst}\ e_0$	Pair Projection: First
		$\texttt{snd}\ e_0$	Pair Projection: Second
		$\texttt{inl}\ e_0$	Sum Type Introduction: Left
		$\texttt{inr}\ e_0$	Sum Type Introduction: Right
		$\texttt{case}\ e_0\ e_1\ e_2$	Sum Type Elimination: Case
		$\texttt{fix}\ \tau\ e_0$	Fixpoint Operator
		$\Lambda\,.\,e_0$	Type Abstraction
		$e_0 \mathbin{@} \tau_0$	Type Application
		$\texttt{nil}\ \tau_0$	Empty List
		$\texttt{nil?}\ e_0$	Empty List Predicate
		$e_1 :: e_2$	List Constructor
		$\texttt{Head}\ e_0$	List Eliminator: Head
		$\texttt{Rest}\ e_0$	List Eliminator: Tail

2.2 Well-Formedness of Types

$$\frac{}{\Delta \vdash \texttt{Unit ty}}\ \text{TY-UNIT} \qquad \frac{}{\Delta \vdash \texttt{Nat ty}}\ \text{TY-NAT} \qquad \frac{}{\Delta \vdash \texttt{Bool ty}}\ \text{TY-BOOL}$$

$$\frac{\Delta \vdash \tau_1\ \texttt{ty} \quad \Delta \vdash \tau_2\ \texttt{ty}}{\Delta \vdash \tau_1 \times \tau_2\ \texttt{ty}}\ \text{TY-PROD} \qquad \frac{\Delta \vdash \tau_1\ \texttt{ty} \quad \Delta \vdash \tau_2\ \texttt{ty}}{\Delta \vdash \tau_1 \oplus \tau_2\ \texttt{ty}}\ \text{TY-SUM} \qquad \frac{\Delta \vdash \tau_1\ \texttt{ty} \quad \Delta \vdash \tau_2\ \texttt{ty}}{\Delta \vdash \tau_1 \to \tau_2\ \texttt{ty}}\ \text{TY-ARR}$$

$$\frac{\Delta(n) = \texttt{Just}\ (\texttt{TVar}\ n)}{\Delta \vdash (\texttt{TVar}\ n)\ \texttt{ty}}\ \text{TY-VAR} \qquad \frac{\Delta, \star \vdash \tau_0\ \texttt{ty}}{\Delta \vdash \forall.\,\tau_0\ \texttt{ty}}\ \text{TY-ALL} \qquad \frac{\Delta \vdash \tau_0\ \texttt{ty}}{\Delta \vdash \texttt{List}\ \tau_0\ \texttt{ty}}\ \text{TY-LIST}$$

2.3 Typing Rules for System F+

2.3.1 Typing Rules for Base Terms

$$\frac{}{\Delta\,;\Gamma \vdash \texttt{unit} : \texttt{Unit}}\ \tau\text{-UNIT} \qquad \frac{}{\Delta\,;\Gamma \vdash \texttt{true} : \texttt{Bool}}\ \tau\text{-TRUE} \qquad \frac{}{\Delta\,;\Gamma \vdash \texttt{false} : \texttt{Bool}}\ \tau\text{-FALSE}$$

$$\frac{\Delta\,;\Gamma \vdash e : \texttt{Nat}}{\Delta\,;\Gamma \vdash \texttt{zero?}\ e : \texttt{Bool}}\ \tau\text{-ZERO?} \qquad \frac{}{\Delta\,;\Gamma \vdash \texttt{zero} : \texttt{Nat}}\ \tau\text{-ZERO} \qquad \frac{\Delta\,;\Gamma \vdash e : \texttt{Nat}}{\Delta\,;\Gamma \vdash \texttt{suc}\ e : \texttt{Nat}}\ \tau\text{-SUC}$$

$$\frac{\Delta\,;\Gamma \vdash e : \texttt{Nat}}{\Delta\,;\Gamma \vdash \texttt{pred}\ e : \texttt{Nat}}\ \tau\text{-PRED} \qquad \frac{\Delta \vdash \tau_0\ \mathsf{ty} \quad \Gamma(n) = \texttt{Just}\ \tau_0}{\Delta\,;\Gamma \vdash \texttt{var}\ n : \tau_0}\ \tau\text{-VAR} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \texttt{Bool} \quad \Delta\,;\Gamma \vdash e_1 : \tau \quad \Delta\,;\Gamma \vdash e_2 : \tau}{\Delta\,;\Gamma \vdash \texttt{if}\ e_0\ e_1\ e_2 : \tau}\ \tau\text{-IF}$$

2.3.2 Typing Rules for Products

$$\frac{\Delta\,;\Gamma \vdash e_1 : \tau_1 \quad \Delta\,;\Gamma \vdash e_2 : \tau_2}{\Delta\,;\Gamma \vdash \langle e_1, e_2\rangle : \tau_1 \times \tau_2}\ \tau\text{-PAIR} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \tau_1 \times \tau_2}{\Delta\,;\Gamma \vdash \texttt{fst}\ e_0 : \tau_1}\ \tau\text{-FST} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \tau_1 \times \tau_2}{\Delta\,;\Gamma \vdash \texttt{snd}\ e_0 : \tau_2}\ \tau\text{-SND}$$

2.3.3 Typing Rules for Functions

$$\frac{\Delta \vdash \tau_1\ \mathsf{ty} \quad \Delta\,;(\Gamma, \tau_1) \vdash e_0 : \tau_2}{\Delta\,;\Gamma \vdash (\lambda\, e_0) : \tau_1 \to \tau_2}\ \tau\text{-LAM} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \tau_1 \to \tau_2 \quad \Delta\,;\Gamma \vdash e_1 : \tau_1}{\Delta\,;\Gamma \vdash \texttt{app}\ e_0\ e_1 : \tau_2}\ \tau\text{-APP} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \tau_0 \quad \Delta\,;(\Gamma, \tau_0) \vdash e_0 : \tau_0}{\Delta\,;\Gamma \vdash \texttt{fix}\ \tau_0\ e_0 : \tau_0}\ \tau\text{-FIX}$$

2.3.4 Typing Rules for Sums

$$\frac{\Delta \vdash \tau_2\ \mathsf{ty} \qquad \Delta\,;\Gamma \vdash e_0 : \tau_1}{\Delta\,;\Gamma \vdash \texttt{inl}\ e_0 : \tau_1 \oplus \tau_2}\ \tau\text{-INL} \qquad \frac{\Delta \vdash \tau_1\ \mathsf{ty} \qquad \Delta\,;\Gamma \vdash e_0 : \tau_2}{\Delta\,;\Gamma \vdash \texttt{inr}\ e_0 : \tau_1 \oplus \tau_2}\ \tau\text{-INR}$$

$$\frac{\Delta\,;\Gamma \vdash e_0 : \tau_1 \oplus \tau_2 \qquad \Delta\,;(\Gamma, \tau_1) \vdash e_1 : \tau_0 \qquad \Delta\,;(\Gamma, \tau_2) \vdash e_2 : \tau_0}{\Delta\,;\Gamma \vdash \texttt{case}\ e_0\ e_1\ e_2 : \tau_0}\ \tau\text{-INR}$$

2.3.5 Typing Rules for Lists

$$\frac{\Delta \vdash \tau_0\ \mathsf{ty}}{\Delta\,;\Gamma \vdash \texttt{nil}\ \tau : \texttt{List}\ \tau_0}\ \tau\text{-NIL} \qquad \frac{\Delta\,;\Gamma \vdash e_1 : \tau_0 \qquad \Delta\,;\Gamma \vdash e_2 : \texttt{List}\ \tau_0}{\Delta\,;\Gamma \vdash e_1 :: e_2 : \texttt{List}\ \tau_0}\ \tau\text{-LCONS}$$

$$\frac{\Delta\,;\Gamma \vdash e_0 : \texttt{List}\ \tau_0}{\Delta\,;\Gamma \vdash \texttt{nil?}\ e_0 : \texttt{Bool}}\ \tau\text{-NIL?} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \texttt{List}\ \tau_0}{\Delta\,;\Gamma \vdash \texttt{Head}\ e_0 : \texttt{Unit} \oplus \tau_0}\ \tau\text{-HEAD} \qquad \frac{\Delta\,;\Gamma \vdash e_0 : \texttt{List}\ \tau_0}{\Delta\,;\Gamma \vdash \texttt{Rest}\ e_0 : \tau_0}\ \tau\text{-REST}$$

2.3.6 Typing Rules for Polymorphism

$$\frac{(\Delta, \star)\,;\texttt{inc}(\Gamma) \vdash e_0 : \tau_0}{\Delta\,;\Gamma \vdash \Lambda\, e_0 : \forall.\, \tau_0}\ \tau\text{-LAM}\mathrm{T} \qquad \frac{\Delta \vdash \tau_2\ \mathsf{ty} \qquad \Delta\,;\Gamma \vdash e_0 : \forall.\, \tau_0}{\Delta\,;\Gamma \vdash e_0\,@\,\tau_2 : \tau_1\,[\,\tau_2\,]}\ \tau\text{-APP}\mathrm{T}$$

2.4 Operational Semantics for System F+

2.4.1 Small-Step Operational Semantics for Base Terms

$$\frac{}{\texttt{zero?}\ (\texttt{zero}) \longmapsto \texttt{true}}\ \text{SS-ZERO?} \qquad \frac{\mathsf{val}_{\texttt{Nat}}\ e_0}{\texttt{zero?}\ (\texttt{suc}\ e_0) \longmapsto \texttt{true}}\ \text{SS-ZERO?-SUC}$$

$$\frac{}{\texttt{if true}\ e_1\ e_2 \longmapsto e_1}\ \text{SS-IF-TRUE} \qquad \frac{}{\texttt{if true}\ e_1\ e_2 \longmapsto e_2}\ \text{SS-IF-FALSE}$$

$$\frac{}{\texttt{pred zero} \longmapsto \texttt{zero}}\ \text{SS-PRED-ZERO} \qquad \frac{\mathsf{val}_{\texttt{Nat}}\ e_0}{\texttt{pred}\ (\texttt{suc}\ e_0) \longmapsto e_0}\ \text{SS-PRED-SUC}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{zero?}\ e_0 \longmapsto \texttt{zero?}\ e_0'}\ \text{SS-ZERO?-STEP} \qquad \frac{e_0 \longmapsto e_0'}{\texttt{suc}\ e_0 \longmapsto \texttt{suc}\ e_0'}\ \text{SS-SUC-STEP}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{pred}\ e_0 \longmapsto \texttt{pred}\ e_0'}\ \text{SS-PRED-STEP} \qquad \frac{e_0 \longmapsto e_0'}{\texttt{if}\ e_0\ e_1\ e_2 \longmapsto \texttt{if}\ e_0'\ e_1\ e_2}\ \text{SS-IF-STEP}$$

2.4.2 Small-Step Operational Semantics for Products

$$\frac{\mathsf{val}_{\tau_1}\ e_1 \qquad \mathsf{val}_{\tau_2}\ e_2}{\texttt{fst}\ \langle e_1,\, e_2\rangle \longmapsto e_1}\ \text{SS-FST} \qquad \frac{\mathsf{val}_{\tau_1}\ e_1 \qquad \mathsf{val}_{\tau_2}\ e_2}{\texttt{snd}\ \langle e_1,\, e_2\rangle \longmapsto e_2}\ \text{SS-SND} \qquad \frac{e_1 \longmapsto e_1'}{\langle e_1,\, e_2\rangle \longmapsto \langle e_1',\, e_2\rangle}\ \text{SS-PAIR-LEFT-STEP}$$

$$\frac{e_2 \longmapsto e_2'}{\langle e_1,\, e_2\rangle \longmapsto \langle e_1,\, e_2'\rangle}\ \text{SS-PAIR-RIGHT-STEP} \qquad \frac{e_0 \longmapsto e_0'}{\texttt{fst}\ e_0 \longmapsto \texttt{fst}\ e_0'}\ \text{SS-FST-STEP}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{snd}\ e_0 \longmapsto \texttt{snd}\ e_0'}\ \text{SS-SND-STEP}$$

2.4.3 Small-Step Operational Semantics for Functions and Recursion

$$\frac{e_0 \longmapsto e_0'}{\texttt{app}\ e_0\ e_1 \longmapsto \texttt{app}\ e_0'\ e_1}\ \text{SS-APP-FUNC} \qquad \frac{e_1 \longmapsto e_1'}{\texttt{app}\ e_0\ e_1 \longmapsto \texttt{app}\ e_0\ e_1'}\ \text{SS-APP-ARG}$$

$$\frac{\mathsf{val}_{\tau_1}\ e_1}{\texttt{app}\ \lambda\,.\,e_0\ e_1 \longmapsto e_0\,[\,e_1\,]}\ \text{SS-APP-BETA} \qquad \frac{\mathsf{val}_{\tau_0}\ e_0}{\texttt{fix}\ \tau_0\ e_0 \longmapsto e_0\,[\,\texttt{fix}\ \tau_0\ e_0\,]}\ \text{SS-FIX-BETA}$$

2.4.4 Small-Step Operational Semantics for Sums

$$\frac{\mathsf{val}_{\tau_1}\ e_0}{\texttt{case}\ (\texttt{inl}\ e_0)\ e_1\ e_2 \longmapsto e_1\,[\,e_0\,]}\ \text{SS-CASE-INL} \qquad \frac{\mathsf{val}_{\tau_2}\ e_0}{\texttt{case}\ (\texttt{inr}\ e_0)\ e_1\ e_2 \longmapsto e_2\,[\,e_0\,]}\ \text{SS-CASE-INR}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{inl}\ e_0 \longmapsto \texttt{inl}\ e_0'}\ \text{SS-INL-STEP} \qquad \frac{e_0 \longmapsto e_0'}{\texttt{inr}\ e_0 \longmapsto \texttt{inr}\ e_0'}\ \text{SS-INR-STEP}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{case}\ e_0\ e_1\ e_2 \longmapsto \texttt{case}\ e_0'\ e_1\ e_2}\ \text{SS-CASE-STEP}$$

2.4.5 Small-Step Operational Semantics for Polymorphism

$$\frac{e_0 \longmapsto e_0'}{e_0 \mathbin{@} \tau_0 \longmapsto e_0 \mathbin{@} \tau_0}\ \text{SS-APPT-FUNC} \qquad \frac{}{(\Lambda\,.\,e_0) \mathbin{@} \tau_0 \longmapsto e_0\,[\,\tau_0\,]}\ \text{SS-APPT-BETA}$$

2.4.6 Small-Step Semantics for Lists

$$\frac{}{\texttt{nil?}\ \texttt{nil}\ \tau_0 \longmapsto \texttt{true}}\ \text{SS-NIL-TRUE} \qquad \frac{}{\texttt{nil?}\ e_1\ ::\ e_2 \longmapsto \texttt{false}}\ \text{SS-NIL-FALSE}$$

$$\frac{}{\texttt{Head}\ (\texttt{nil}\ \tau_0) \longmapsto \texttt{inl}\ \texttt{unit}}\ \text{SS-HEAD-NIL} \qquad \frac{\mathsf{val}_{\tau_0}\ e_1 \qquad \mathsf{val}_{(\texttt{List}\ \tau_0)}\ e_2}{\texttt{Head}\ (e_1\ ::\ e_2) \longmapsto \texttt{inr}\ e_1}\ \text{SS-HEAD-LCONS}$$

$$\frac{}{\texttt{Rest}\ (\texttt{nil}\ \tau_0) \longmapsto \texttt{inl}\ \tau_0}\ \text{SS-REST-NIL} \qquad \frac{\mathsf{val}_{\tau_0}\ e_1 \qquad \mathsf{val}_{(\texttt{List}\ \tau_0)}\ e_2}{\texttt{Rest}\ (e_1\ ::\ e_2) \longmapsto \texttt{inr}\ e_2}\ \text{SS-REST-LCONS}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{nil?}\ e_0 \longmapsto \texttt{nil?}\ e_0'}\ \text{SS-NIL-STEP} \qquad \frac{e_1 \longmapsto e_1'}{e_1\ ::\ e_2 \longmapsto e_1'\ ::\ e_2}\ \text{SS-CONS-HEAD-STEP}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{Head}\ e_0 \longmapsto \texttt{Head}\ e_0'}\ \text{SS-HEAD-STEP} \qquad \frac{e_2 \longmapsto e_2'}{e_1\ ::\ e_2 \longmapsto e_1\ ::\ e_2'}\ \text{SS-CONS-REST-STEP}$$

$$\frac{e_0 \longmapsto e_0'}{\texttt{Rest}\ e_0 \longmapsto \texttt{Rest}\ e_0'}\ \text{SS-REST-STEP}$$

2.5 Value Judgment for System F+

$$\frac{}{\mathsf{val}_{\texttt{Bool}}\ \texttt{true}}\ \text{V-TRUE} \qquad \frac{}{\mathsf{val}_{\texttt{Bool}}\ \texttt{false}}\ \text{V-FALSE} \qquad \frac{}{\mathsf{val}_{\texttt{Nat}}\ \texttt{zero}}\ \text{V-ZERO} \qquad \frac{\mathsf{val}_{\texttt{Nat}}\ e_0}{\mathsf{val}_{\texttt{Nat}}\ (\texttt{suc}\ e_0)}\ \text{V-SUC}$$

$$\frac{}{\mathsf{val}_{\texttt{Unit}}\ \texttt{unit}}\ \text{V-UNIT} \qquad \frac{\mathsf{val}_{\tau_1}\ e_1 \qquad \mathsf{val}_{\tau_2}\ e_2}{\mathsf{val}_{\tau_1 \times \tau_2}\ \langle e_1,\, e_2 \rangle}\ \text{V-PAIR} \qquad \frac{\mathsf{val}_{\tau_1}\ e_0}{\mathsf{val}_{\tau_1 \oplus \tau_2}\ \texttt{inl}\ e_0}\ \text{V-INL} \qquad \frac{\mathsf{val}_{\tau_2}\ e_0}{\mathsf{val}_{\tau_1 \oplus \tau_2}\ \texttt{inr}\ e_0}\ \text{V-INR}$$

$$\frac{}{\mathsf{val}_{\tau_1 \to \tau_2}\ \lambda\,.\,e_0}\ \text{V-LAM} \qquad \frac{}{\mathsf{val}_{\forall.\,\tau_0}\ \Lambda\,.\,e_0}\ \text{V-LAMT} \qquad \frac{}{\mathsf{val}_{\tau_0}\ \texttt{nil}\ \tau_0}\ \text{V-NIL} \qquad \frac{\mathsf{val}_{\tau_0}\ e_1 \qquad \mathsf{val}_{\texttt{List}\ \tau_0}\ e_2}{\mathsf{val}_{\texttt{List}\ \tau_0}\ e_1\ ::\ e_2}\ \text{V-CONS}$$

3 Theorems for System F+

Theorem 3.1 (Type Safety)**.** *System F+ is type safe.*

Type safety is the ultimate property that we wish to prove about System F+. Being type safe implies that well-typedness is preserved as an invariant during the evaluation of System F+ programs. In other words, the small-step operational semantics respect the typing rules; hence, if the input programs type check, then the final evaluation result will be of the same type.

To prove type safety, we must prove its two properties: progress and preservation. Progress states that

a well-typed expression will not get stuck. Preservation states that making progress will not change the type of the expression. These two lemmas will be discussed in the following section.

However, type safety comes with its own constraint; to be specific, being type safe doesn't mean all input program can terminate because the program may loop forever. In the original System F, we can prove its termination using logical relation. For System F+, this is impossible because we introduced the fixed-point operators, which makes it very convenient to create programs that may diverge. So, this project only focuses on the type safety of System F+.

3.1 Progress

Progress, in effect, states that a well-typed term is either a value or takes a step to another expression. This means that no well-typed term gets stuck when evaluating.

Lemma 3.1 (Progress)**.** *If* $\cdot\,;\cdot \vdash e_0 : \tau_0$*, then either* $\mathsf{val}_{\tau_0}\ e_0$ *or* $e_0 \longmapsto e_0'$*.*

Proof. By rule induction on $\cdot\,;\cdot \vdash e_0 : \tau_0$. □

Some materials [2] highlight that proving the progress lemma requires the canonical forms lemma, which confirms that values structurally match their types. For example, it ensures that any value v of type $\tau_1 \to \tau_2$ must take the form of a function abstraction $\lambda\,.\,e_0$. This requirement turns out to be redundant for our system because our $\mathsf{val}_\tau\ e$ judgment encodes canonical forms within its definition.

3.2 Preservation

Preservation states that if we have a well-typed term of some type and we know that it steps to another expression, then it does not change types when stepping.

Lemma 3.2 (Preservation)**.** *If* $\cdot\,;\cdot \vdash e_0 : \tau_0$ *and* $e_0 \longmapsto e_0'$*, then* $\cdot\,;\cdot \vdash e_0' : \tau_0$*.*

Proof. By rule induction on $e_0 \longmapsto e_0'$. □

Note that, in order to prove preservation, we need several auxiliary lemmas about the substitution of variables in terms, type environments, and type variables. These three lemmas are proved in §3.2.1 and §3.2.2.

3.2.1 Substitution Lemma for Term Variables

Theorem 3.2. *If* $\cdot\,;(\cdot, \tau_1) \vdash e_0 : \tau_2$ *and* $\cdot\,;\cdot \vdash v_0 : \tau_1$*, then* $\Delta\,;\Gamma \vdash e_0\,[\,v_0\,] : \tau_2$*.*

The substitution lemma is the pillar for the proof of the preservation lemma because it illustrates that β-reduction by SS-APP-BETA and SS-FIX-BETA always preserves type safety.

Unfortunately, adopting De Bruijn indices makes it impossible for us to abuse notation like we would with a pen-and-paper proof. As a result, instead of directly proving this substitution lemma by induction, we have to prove a more complicated, index-aware version, and the straightforward one becomes its corollary. Detailed explanations are omitted in this document; if the reader is interested in the proof detail, please refer to the Agda source code.

3.2.2 Substitution Lemma for Type Variables

There are two substitution lemmas that must work coherently to guarantee that the well-typedness is preserved. On the one hand, we need to ensure that substituting a type variable in a type still produces a valid type, which is specified by the following lemma.

Lemma 3.3. *If* $\Delta \leq i$ *and* $\Delta \vdash \tau_1\ \mathsf{ty}$*, then* $\Delta \vdash \tau_1 [\tau_2]_i\ \mathsf{ty}$*.*

To be more specific, this lemma showcases that substituting a type variable not in the type variable context Δ would be a no-op. This lemma tells us that, if Δ is empty, any substitution is a no-op, which substantially simplifies the process.

On the other hand, the following lemma shows well-typedness of polymorphic terms: we can instantiate it by providing a concrete type.

Lemma 3.4. *If* $\cdot \vdash \tau_1\ \mathsf{ty}$ *and* $(\cdot, \star) ; \cdot \vdash e_0 : \tau_2$*, then* $\cdot ; \cdot \vdash e_0 @ \tau_1 : \tau_2 [\tau_1]$*.*

Similar to the substitution lemma for term variables, we can only prove its index-aware counterpart and extract this straightforward version as the corollary. More details are included in the Agda source code.

3.2.3 Weakening Lemmas

Lemma 3.5. *If* $\cdot ; \cdot \vdash e_0 : \tau_0$*, then* $\Delta ; \Gamma \vdash e_0' : \tau_0$ *holds for any* Δ *and* Γ*.*

The weakening lemma is a summary of the two weakening lemmas for term variables and type variables. Weakening roughly states that, if we have a term $e : \tau$ in some context Γ, extending the context to $\Gamma, x : \tau'$ preserves e's type. Note that this applies to environments for both terms and types.

The motivation to introduce the weakening lemma is that, when applying the substitution lemmas, we need to adjust the type variable context Δ and the term variable context Γ for the substituted value so that it can match the function body.

4 Interpreter

It is helpful to be able to evaluate programs/expressions in our language to ensure that the type safety behavior works as intended. Unfortunately, each term requires a type, and both the term and the type are manually written, which quickly becomes tedious for even moderately-sized programs. Plus, the programmer must remember that terms using other terms defined outside the current environment need to be weakened to a non-empty context. To remove some of the tedium, we built an interpreter/-translator in the Racket programming language, which receives an expression in an s-expression-based language. The grammar of the s-expression-based language is defined below.

When the programmer runs the interpreter, it produces both an Agda term and its type. These can then be copied and pasted into our Agda file and type-checked. Because our language supports the fixed-point operator `fix`, we cannot prove strong normalization/termination. Instead, we use a *step-indexed evaluator* to return either the result of the expression or "Nothing" if the expression does not finish evaluating in N steps [1].

4.1 Grammar for Interpreter

Types	τ ::=	Nat	Natural Numbers
		Bool	Boolean
		X	Type Variables
		$(\forall\ X\ \tau)$	Universal Type
		$(\tau_1 \Rightarrow \tau_2)$	Function Type
		$(\mathsf{List}\ \tau)$	List Type

Terms	e ::=	x	Term Variables
		X	Type Variables
		n	Natural Number Literals
		b	Boolean Literals
		(`null?` e)	List Null Test
		(`add1` e)	Successor
		(`sub1` e)	Predecessor
		(`zero?` e)	Zero Test
		(`first` e)	First Element
		(`rest` e)	Rest of List
		(`lcons` e_1 e_2)	List Construction
		(`let` $((x : \tau)\ e_1)\ e_2$)	Typed Let Binding
		(`lambda` $(x : \tau)\ e$)	Typed Lambda
		(`if` e_1 e_2 e_3)	Conditional
		(`case` e_1 e_2 $((x : \tau)$ `=>` $e_3)$)	Case Analysis
		$(e_1\ e_2)$	Term Application
		(e `@` τ)	Type Application
		(`fix` τ x e)	Fixed Point
		$(\forall X\ e)$	Type Abstraction

4.2 Polymorphic List Map Function

```
(define map-example
  `(fix (∀ A (∀ B ((A ⇒ B) ⇒ ((List A) ⇒ (List B)))))
     map
     (lambda (f : (A ⇒ B))
       (lambda (ls : (List A))
         (if (null? ls) ls
             (case (first ls)
               ls
               ((h : A) => (lcons (f h) ((((append @ A) @ B) f) (rest ls)))))))))))
```

5 Summary

We develop a formalism of System F+ in the Agda language, including its syntax, type system, and reduction rules. In addition, we prove progress and preservation lemmas to demonstrate the type safety of System F+. To facilitate the proof, we further build a racket interpreter to construct Agda terms automatically. This project turns out to be not only a good opportunity to horn our Agda skills, but also a rewarding effort to formalize a language without any ambiguity.

References

[1] Nada Amin and Tiark Rompf. Type soundness proofs with definitional interpreters. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, POPL '17, page 666–679, New York, NY, USA, 2017. Association for Computing Machinery.

[2] Carlo Angiuli. Programming language foundations. Lecture notes for CSCI-B522, Indiana University, 2025. `https://www.carloangiuli.com/courses/b522-sp25/index.html`.

[3] Nicolaas Govert De Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. In *Indagationes mathematicae (proceedings)*, volume 75, pages 381–392. Elsevier, 1972.

[4] Jean-Yves Girard. Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur. 1972.

[5] Rob Nederpelt and Herman Geuvers. *Type theory and formal proof: an introduction*. Cambridge University Press, 2014.